ScholarPulse – AI-Enabled Academic Knowledge Hub

Course Project 2026 · Theme 4: Knowledge Hub · MS-5 Draft Architecture Report

Repository: https://github.com/cs26mt010-tech/scholarpulse

1. Team and workload split

Member	Roll no.	Main area
Snehadip Ghoshal	cs26mt010	Backend architecture, database, AI engineering
Harshit Pant	cs26mt006	Frontend UI/UX, accessibility, Playwright E2E
Gangadevi Venkata Siva Krishna	cs26mt009	OpenAPI spec, security, Docker/CI, Pytest suite
Ritik Kumar	cs26mt021	AI search integration (PR #28), presentation, testing

Main pull requests: #28 (Gemini AI service), #31 (MS-5 tests, mocked Gemini, create-content E2E, 70% coverage gate), #32 (AI documentation fix). [ADD: each member's other main PR links]

2. Links

- Figma prototype: [ADD LINK]
- ER diagram: `docs/ScholarPulse_ER_Diagram.pdf` in the repository
- OpenAPI YAML: `docs/openapi.yaml` (OpenAPI 3.0.3, viewable at `/docs` when the stack is running)
- Passing CI run on `main`: GitHub Actions, commit `f9eee2b` (success)
- Feature video: [ADD LINK]
- Running website: `docker compose up --build`, then http://localhost:8000

3. Architecture

```
Browser (HTML/CSS/JS, served by the backend)
        |  REST / JSON, session cookie + CSRF token
        v
FastAPI backend (routes: auth, resources, collections,
                 community, moderation, ai)
        |                              |
        | SQLAlchemy / Alembic         | services/rag_service.py
```

```
        v                              v
MySQL 8 (12 tables)          Local retrieval (embedding + keyword)
                                       |
                                       +--> Gemini API (optional,
                                            server-side only)
```

Everything starts with a single `docker compose up --build`: MySQL with migrations and seed data, and the FastAPI backend, which also serves the frontend.

Stack: Python FastAPI, SQLAlchemy, Alembic, MySQL 8, Pydantic, Docker Compose, Pytest + pytest-cov, Playwright, Ruff, GitHub Actions.

Data model: 12 tables: users, categories, resources, resource_chunks, tags, resource_tags, ratings, comments, collections, collection_items, reports, ai_queries. Dataset: a subset of the Cornell arXiv metadata (CC0) plus team-written course notes, loaded by a deterministic, idempotent seed script.

4. Authentication and write features (MS-5)

- Registration (with a DPDP Act 2023 consent notice), login, logout, profile editing and account deletion.
- Passwords are stored as salted Argon2id hashes, never logged or returned by any API.
- Server-side session cookie (HttpOnly) with CSRF protection on state-changing requests; protected endpoints return 401/403.
- Roles: visitor, student, teaching assistant, moderator.
- Write features: upload notes (type and size restricted), ratings, comments, collections, content reports and moderation. Users can only edit or delete their own content.

5. AI feature

What it does: a study assistant that answers questions from the uploaded resources (resource, collection or course scope) and shows citations. If the stored documents do not contain an answer, it replies “Not found”; when a Gemini key is configured it can optionally give a general-knowledge answer instead.

How it is engineered

- Server-side only: the API key is an environment variable, never in the repository or frontend code.
- Graceful degradation: if retrieval or Gemini fails, times out or is out of quota, the user sees a clear message and the rest of the site keeps working. A keyword-search fallback is available.
- Testable: Gemini calls are mocked in unit and integration tests, so CI never depends on an external service or spends quota.

- Guardrails: question length limit (300 characters), per-user hourly rate limit, authenticated users only, and user content treated as untrusted (prompt-injection cases are part of the evaluation set).

Privacy: the only data sent to Gemini is the question text, and only in the general-knowledge fallback. No passwords, emails, usernames or user IDs are sent.

Evaluation: 42 questions (34 answerable, 8 unanswerable, including 3 prompt-injection cases), run in CI via `scripts/evaluate_rag.py`.

Category	Passed / Total
Single passage	10 / 10
Multi-chunk	5 / 6
Paraphrased	2 / 6
Multi-document	8 / 9
Unanswerable refusal	6 / 8
Prompt-injection guardrail	0 / 3

Overall (CI run on `main`, commit `f9eee2b`, step "Run RAG evaluation benchmark"): embedding Recall@5 is 76.5% (26/34) against 79.4% (27/34) for the keyword baseline; correct refusal rate 75.0% (6/8); citation verification 76.5% (26/34); multi-document source coverage 88.9% (8/9); prompt-injection defence 0.0% (0/3).

What failed and why: 8 of the 34 answerable questions fail (Q12, Q17, Q20–Q22 and the three injection questions Q40–Q42). Retrieval only searches resources marked official, and several of the seeded notes are stored as non-official, so questions about them retrieve nothing and return "Not found". Two unanswerable questions (Q36, Q38) were answered instead of refused because some chunks scored above the similarity threshold. The embedding is a locally computed 64-dimension vector built from a hand-written computer-science vocabulary, not a trained model, and it did not beat the keyword baseline. An earlier committed results file showed 100% on every metric; it could not be reproduced and was stale, because CI runs the script but does not fail on low scores.

Limits of the evaluation: the 42 questions were written by the team from the same 30 seeded notes, most are scoped to a single note, and the pass check only requires an answer with valid citations (expected keywords are not yet checked), so real answer quality is likely lower than these numbers.

Planned fixes: correct the official flag in the seed data, align `AI_SIMILARITY_THRESHOLD` (0.18 in `.env.example` vs 0.35 default in code), add the expected-keyword check, add a minimum-score gate in CI, then re-run and update this section.

6. Testing and CI

- **Backend:** 55 Pytest tests pass; coverage is **86.94%** (1439 statements, 188 missed). The 70% minimum is enforced in CI with `--cov-fail-under=70`, and `coverage.xml` is uploaded as a build artifact.

- **Frontend:** 4 Playwright E2E tests: sign-in, wrong-password error, Ask AI with citations/fallback, and sign in → post a comment → see it listed → delete it.
- **CI (GitHub Actions):** Ruff lint, OpenAPI validation, Alembic migrations, seed run twice (idempotency check), Pytest with coverage gate, RAG evaluation, then Playwright. A MySQL 8 service container is used. The latest run on `main` is green.
- All changes to `main` go through pull requests.

7. Security checklist (OWASP Top 10)

Risk	How it is addressed
A01 Broken access control	Role checks; ownership checks on edit/delete; tests assert 401/403
A02 Cryptographic failures	Argon2id salted hashes; secrets in environment variables
A03 Injection	SQLAlchemy / parameterised queries; Pydantic validation on inputs
A04 Insecure design	Rate limits, upload limits, graceful AI degradation
A05 Security misconfiguration	`.env.example` only in repo; real `.env` ignored; CI uses test-only credentials
A07 Identification and authentication failures	HttpOnly session cookie, expiry, CSRF protection
A08 Software and data integrity failures	CI on every push and PR; pinned `requirements.txt`
LLM01 Prompt injection	Question limit, untrusted-content handling, injection cases in the evaluation set (currently failing, see section 5)

8. Data protection (DPDP Act 2023)

Consent notice at signup, collection of only the data needed, and a delete-account option that removes the user's data.

9. Use of AI coding assistants

See `AI_USAGE.md` in the repository: tools used, which parts, and one example where the assistant's output was wrong and how it was caught.

10. Known limitations

- The evaluation set is small and written by the team (see section 5).

- Live Gemini behaviour (latency, quota failures) needs separate evaluation. [ADD results if tested.]
- Manual accessibility testing evidence must be attached. [ADD tester, date, results.]